CaptionAI: Credit Card Approval Prediction

Project Description:

Credit Card Approval Prediction is a machine learning application that helps financial institutions determine whether a customer's credit card application should be approved or rejected. The system analyzes customer information such as age, income, employment status, credit history, loan amount, and other financial attributes to make accurate predictions. This project demonstrates the use of data preprocessing, exploratory data analysis, machine learning algorithms, model evaluation, and deployment using Flask.

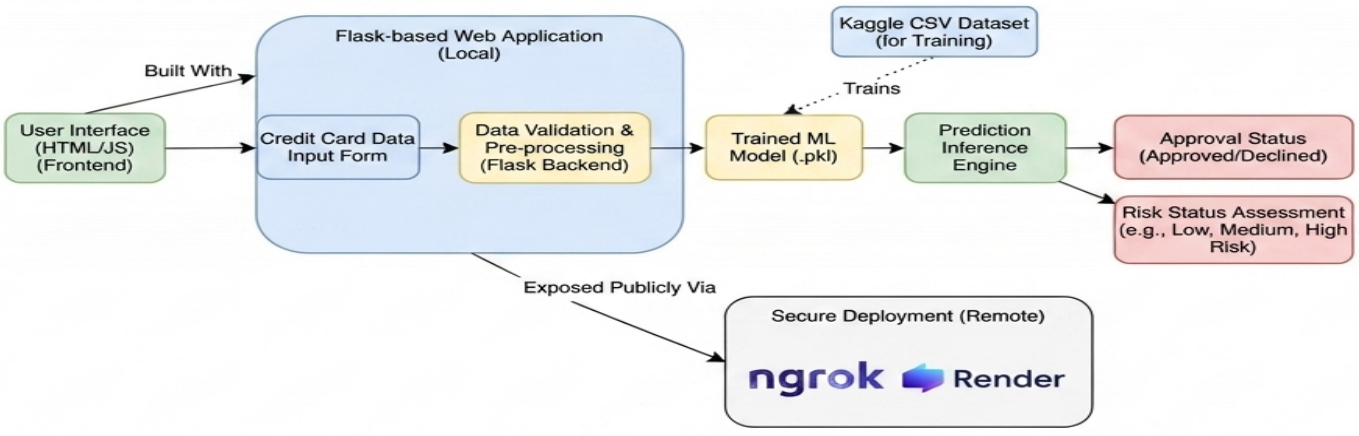
Scenarios

Scenario 1: A customer submits a credit card application by providing personal and financial information. The system processes the details and predicts whether the application is likely to be approved, enabling faster and more reliable decision-making

Scenario 2: A bank officer reviews multiple applications daily. The prediction model assists by identifying eligible applicants based on historical data, reducing manual effort and improving consistency.

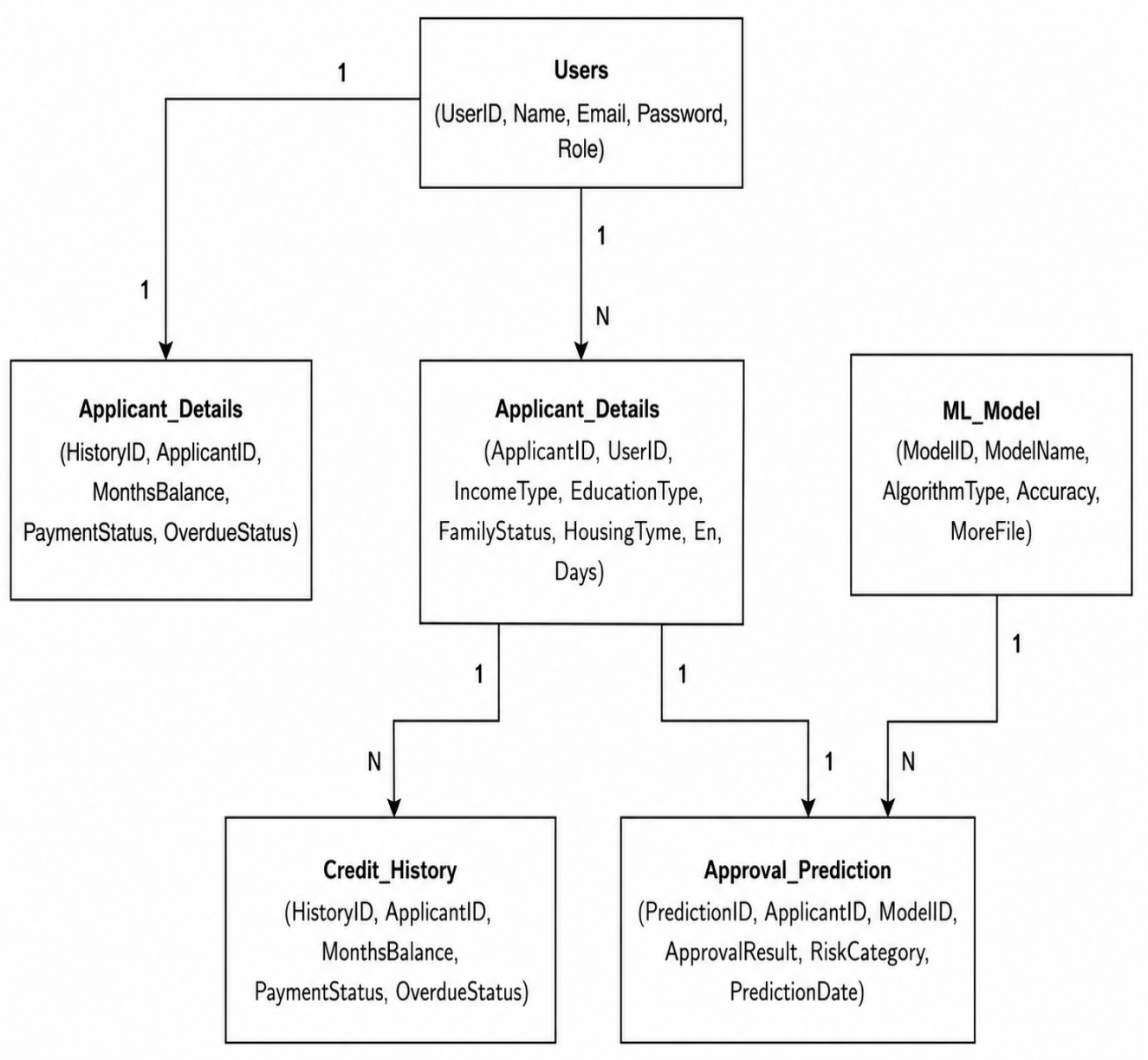
Scenario 3: The application provides prediction results along with important factors influencing the decision. This helps financial institutions understand applicant risk and maintain transparency in the approval process.

Technical Architecture:



Description: The Credit Card Approval Prediction application uses a modular three-tier architecture to deliver rapid, ML-driven predictions. The frontend provides a responsive user interface built with HTML and CSS, while the Flask-based backend orchestrates data validation and feature preprocessing. It interfaces with a Scikit-Learn trained model (Logistic Regression / Random Forest) to generate tailored credit approval predictions, with deployment managed locally via Flask and exposed publicly through Ngrok.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)



Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **Scikit-Learn for Machine Learning:** Scikit-Learn Documentation
- **Pandas & NumPy for Data Analysis:** Pandas Documentation
- **Matplotlib & Seaborn for Visualization:** Matplotlib Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency:** Python Documentation
- **Version Control with Git:** Git Documentation
- **Ngrok for Public Depolyment : Ngrok Documentation**

Project Workflow:

Milestone 1: Data Collection and Environment Setup

- **Activity 1.1:** Set up the Python virtual environment and install all required libraries (Flask, Scikit-Learn, Pandas, NumPy, Matplotlib, Seaborn).
- **Activity 1.2:** Acquire and load the credit card application dataset. Perform initial data inspection to understand feature types, missing values, and class distribution.
- **Activity 1.3:** Define the application architecture, detailing interactions between the frontend, Flask backend, and ML model.
- **Activity 1.4:** Configure the project directory structure including folders for templates, static assets (CSS, JS), plots, and main application files (app.py, requirements.txt).

Milestone 2: Exploratory Data Analysis

. Activity 2.1: Clean the dataset by handling missing values and removing duplicate records and to generate descriptive statictics for all.

. Activity 2.2: Convert categorical features into numerical values using suitable encoding techniques and to produce visuzilation identifying patterns and outliers etc..

. Activity 2.3: Perform Exploratory Data Analysis (EDA) using graphs such as histograms, count plots, box plots, and correlation heatmaps

Milestone 3: Machine Learning Model Development

- **Activity 3.1:** Preprocess the dataset by handling missing values, encoding categorical variables, and scaling numerical features using Scikit-Learn pipelines.
- **Activity 3.2:** Train and evaluate classification models to predict credit card approval. Select the best-performing model based on accuracy, precision, recall, and F1-score.

Milestone 4: Web Application Development (app.py)

- **Activity 4.1:** Write the main application logic in main.py, establishing Flask routes for the home page and prediction endpoint, and integrating ML model inference.

Milestone 5: Frontend Development

- **Activity 5.1:** Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout for applicant data entry.
- **Activity 5.2:** Create dynamic templates using Flask's render_template to display prediction results and EDA visualizations based on user interactions.

Milestone 6: Deployment

- **Activity 6.1:** Prepare the application for local deployment by configuring the server environment and verifying all dependencies are correctly installed.
- **Activity 6.2:** Deploy the application publicly using Ngrok to expose the local Flask server over a secure public URL.

Milestone 7: Conclusion

Milestone 1: Data Collection and Environment Setup

This milestone focuses on collecting the credit card approval dataset and preparing the development environment for the project. It includes downloading the dataset, installing the required software and Python libraries, organizing the project structure, and performing initial data exploration. These activities ensure that the system is ready for data preprocessing and machine learning model development

Activity 1.1: Set Up the Development / virtual Environment

- Install Python and Pip: Ensure Python 3.12+ is installed on your machine along with pip for dependency management.
- Create a Virtual Environment: Set up a virtual environment using venv to isolate project dependencies:

```
PS C:\Users\P YUVA PRIYA\OneDrive\Desktop\Credit Card
Project> >python -m venv myenv
PS C:\Users\P YUVA PRIYA\OneDrive\Desktop\Credit Card
Project> >" PS C:\Users\P YUVA
PRIYA\OneDrive\Desktop\Credit Card Project>
\venv\Scripts\activate"
PS C:\Users\P YUVA PRIYA\OneDrive\Desktop\Credit Card
Project> > pip install -r requirements.txt
```

- Install Flask:

```
PS C:\Users\P YUVA PRIYA\OneDrive\Desktop\Credit Card Project> > pip install
Flask==3.0.3
```

- Install Scikit-Learn:

```
PS C:\Users\P YUVA PRIYA\OneDrive\Desktop\Credit Card
Project> > pip install scikit-learn
```

- Install Pandas and NumPy:

```
PS C:\Users\P YUVA PRIYA\OneDrive\Desktop\Credit Card
Project> > pip install pandas numpy
```

- Install Matplotlib and Seaborn:

```
PS C:\Users\P YUVA PRIYA\OneDrive\Desktop\Credit Card
Project> > pip install matplotlib seaborn
```

- Set Up Application Structure: Create the project directory structure including folders for templates, static files (CSS, JS), plots, and main application files (app.py, requirements.txt).

```
/CREDIT CARD PROJECT
  /data
    - creditcard_data.csv (Dataset file)
  /models
    - card_model.joblib (Trained model)
    - feature_names.joblib (Model feature names)
  /static
    - script.js (Frontend script)
    - style.css (Stylesheet)
  /templates
    - index.html (Webpage template)
  - app.py (Main application file)
  - notebook.ipynb (Jupyter notebook)
  - perf_test.py (Performance testing script)
  - requirements.txt (Python dependencies)
  - run_public.py (Public run script)
  - train.py (Training script)
```

Activity 1.2: Load and Inspect the Dataset

- **Understand the Dataset:** Review the structure and format of the credit card application dataset, including feature names, data types, and the binary target variable (Approved / Rejected).
- **Initial Data Inspection:** Use Pandas to load the dataset and run .info(), .describe(), .value_counts() and .isnull().sum() to understand the shape, missing values, and class balance.
- **Identify Preprocessing Needs:** Document which features require encoding, scaling, or imputation based on the initial inspection

```
df.info()

class 'pandas.DataFrame'>
angeIndex: 9709 entries, 0 to 9708
ata columns (total 20 columns):
#   Column             Non-Null Count  Dtype
--  ------             --------------  -----
0   ID                 9709 non-null   int64
1   Gender             9709 non-null   int64
2   Own_car            9709 non-null   int64
3   Own_property       9709 non-null   int64
4   Work_phone         9709 non-null   int64
5   Phone              9709 non-null   int64
6   Email              9709 non-null   int64
7   Unemployed         9709 non-null   int64
8   Num_children       9709 non-null   int64
9   Num_family         9709 non-null   int64
10  Account_length     9709 non-null   int64
11  Total_income       9709 non-null   float64
12  Age                9709 non-null   float64
13  Years_employed     9709 non-null   float64
14  Income_type        9709 non-null   str
15  Education_type     9709 non-null   str
16  Family_status      9709 non-null   str
17  Housing_type       9709 non-null   str
18  Occupation_type    9709 non-null   str
```

```
df.isnull().sum()

ID                 0
Gender             0
Own_car            0
Own_property       0
Work_phone         0
Phone              0
Email              0
Unemployed         0
Num_children       0
Num_family         0
Account_length     0
Total_income       0
Age                0
Years_employed     0
Income_type        0
Education_type     0
Family_status      0
Housing_type       0
Occupation_type    0
Approved           0
dtype: int64
```

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components such as the frontend (HTML, CSS, JavaScript), Flask backend, and Scikit-Learn ML model integration.
- **Detail Frontend Functionality:** Outline how users enter applicant information and submit it for prediction through the web interface.
- **Outline Backend Responsibilities**: Specify how the Flask backend handles incoming POST requests to /predict, validates and preprocesses user input, runs inference through the trained model, and returns a JSON prediction result.
- **Describe ML Integration Points:** Define how the backend loads the serialized model at startup and uses it to generate predictions from preprocessed applicant feature vectors.

Activity 1.4: Set Up the Project Directory Structure

- **Create the Folder Layout:** Set up the Credit Card Approval Prediction/ root with the app,notebook subdirectory containing models/card_model.joblib, static/ (with css/ and js/ subdirectories), and templates/HTML.
- **Create Core Files:** Initialise app.py as the Flask application entry point, requirements.txt listing all dependencies, and index.html as the primary HTML template.

Milestone 2: Exploratory Data Analysis

Milestone 2 focuses on understanding the credit card dataset through statistical analysis and visualization. The outputs — descriptive statistics and plots — are saved to the app/plots/ directory and are subsequently made available through the web interface for stakeholder review.

Activity 2.1: Generate Descriptive Statistics

- Compute summary statistics for all numerical features (mean, standard deviation, min, max, quartiles) using Pandas. describe().
- Compute frequency counts and mode for all categorical features (e.g., occupation, marital status) using .value_ counts ().
- Save the combined statistical summary to notebook for reference and reporting.

```
# Load the dataset
import pandas as pd
df = pd.read_csv('data/creditcard_data.csv')
print("Dataset loaded.Number of rows:", len(df))
```

```
Dataset loaded.Number of rows: 9709
```

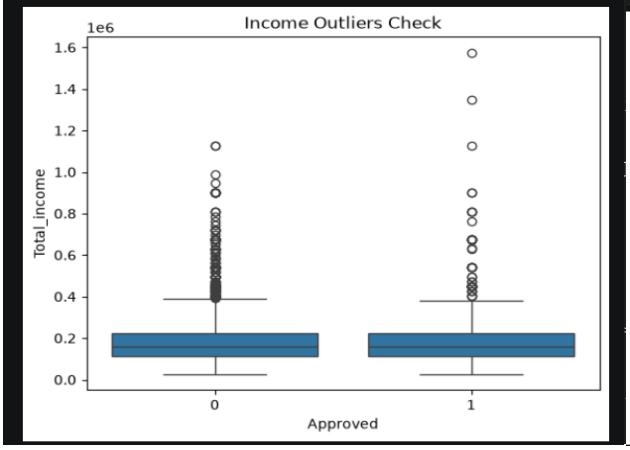
```
df.describe()
```

	ID	Gender	Own_car	Own_property	Work_phone	Phone	Email	Unemployed	Num_children	Num_family	Account_length	Total_income	Age	Years_employed	Target
count	9.709000e+03	9709.000000	9709.000000	9709.000000	9709.000000	9709.000000	9709.000000	9709.000000	9709.000000	9709.000000	9709.000000	9.709000e+03	9709.000000	9709.000000	9709.000000
mean	5.076105e+06	0.348749	0.367700	0.671542	0.217427	0.287671	0.087548	0.174683	0.422804	2.182614	27.270059	1.812282e+05	43.784093	5.664730	0.132145
std	4.080270e+04	0.476599	0.482204	0.469677	0.412517	0.452700	0.282650	0.379716	0.767019	0.932918	16.648057	9.927731e+04	11.625768	6.342241	0.338666
min	5.008804e+06	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	2.700000e+04	20.504186	0.000000	0.000000
25%	5.036955e+06	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	2.000000	13.000000	1.125000e+05	34.059563	0.928150	0.000000
50%	5.069449e+06	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	2.000000	26.000000	1.575000e+05	42.741466	3.761884	0.000000
75%	5.112988e+06	1.000000	1.000000	1.000000	0.000000	1.000000	0.000000	0.000000	1.000000	3.000000	41.000000	2.250000e+05	53.567151	8.200031	0.000000
max	5.150479e+06	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	19.000000	20.000000	60.000000	1.575000e+06	68.863837	43.020733	1.000000

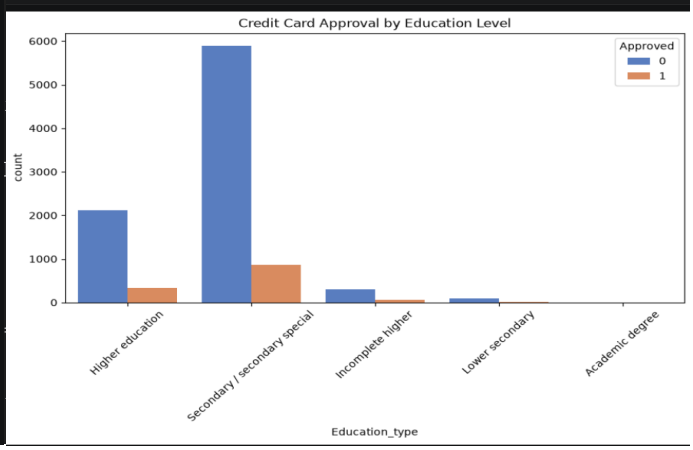
Activity 2.2: Univariate Analysis

- Generate a distribution plot for the Income feature and save in notebook as univariate_income.png. Identify skewness and the presence of high-income outliers that may require log transformation.
- Generate a bar chart for the Education feature and save in notebook as univariate_education.png. Assess class frequencies and identify underrepresented education categories.
- Use Matplotlib and Seaborn for all plot generation, applying consistent styling (figure size, axis labels, titles, and color palettes).

```
sns.boxplot(x='Approved', y='Total_income', data=df)
plt.title('Income Outliers Check')
plt.show()
```



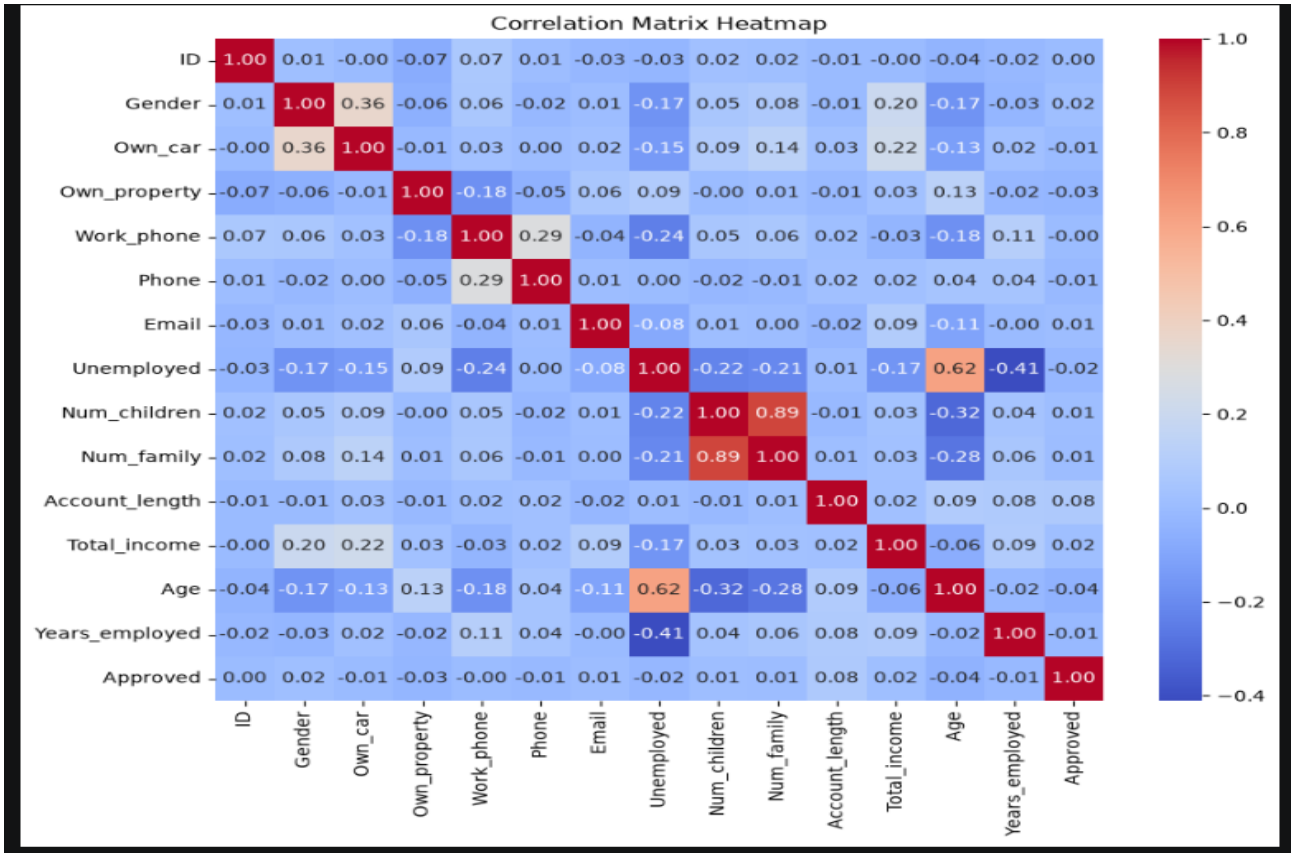

```
plt.figure(figsize=(10, 5))
sns.countplot(x='Education_type', hue='Approved', data=df, palette='muted')
plt.xticks(rotation=45)
plt.title('Credit Card Approval by Education Level')
plt.show()
```



Activity 2.3: Multivariate Correlation Analysis

- Compute the correlation matrix for all numerical features using numeric_ df_ corr().
- Generate a heatmap using Seaborn and save it as notebook/multivariate_correlation.png. Annotate each cell with the correlation coefficient for clarity.
- Analyze the heatmap to identify features with strong positive or negative correlations with the approval target variable, informing feature selection for the ML model.

```
# Select only numeric columns for correlation calculation
numeric_df = df.select_dtypes(include=[np.number])
plt.figure(figsize=(10, 8))
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix Heatmap')
plt.show()
```



Milestone 3: Machine Learning Model Development

Milestone 3 focuses on building and evaluating the predictive model. The dataset is preprocessed using Scikit-Learn pipelines, classification models are trained and compared, and the best-performing model is serialized for integration with the Flask backend.

Activity 3.1: Data Preprocessing

- **Handle Missing Values:** Apply median imputation for numerical features and most-frequent imputation for categorical features using SimpleImputer.
- **Encode Categorical Variables:** Apply LabelEncoder or OneHotEncoder to transform categorical features (e.g., Gender, Occupation, Marital Status) into numerical representations.
- **Scale Numerical Features:** Apply StandardScaler to normalize numerical features and ensure consistent feature magnitude for distance-based models.
- **Split the Dataset:** Partition the dataset into training (80%) and test (20%) sets using train_test_split with stratification to preserve class balance.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

# Updated column names to match your dataset exactly
categorical_cols = ['Gender', 'Income_type', 'Education_type']

for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

print("Encoding complete. Categorical features converted to numbers.")
df.head()
```

```
    X = df.drop(columns=['Approved'])
    y = df['Approved']
```

```
    from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    print(f"Train size: {X_train.shape[0]}, Test size: {X_test.shape[0]}")
```

```
Train size: 7767, Test size: 1942
```

Activity 3.2: Model Training and Evaluation

- **Train Classification Models:** Train multiple classifiers — including Logistic Regression and Random Forest — on the preprocessed training set.
- **Evaluate Model Performance:** Assess each model using accuracy, precision, recall, and F1score on the held-out test set. Use cross-validation to obtain robust performance estimates.
- **Select the Optimal Model:** Choose the model with the best balance of predictive accuracy and generalizability for deployment.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

# 1. Drop the ID column first so it isn't included in one-hot encoding
if 'ID' in df.columns:
    df = df.drop(columns=['ID'])

# 2. Define target column
target_col = 'Approved'

# 3. Use One-Hot Encoding to automatically convert all categorical variables to clean 1s and 0s
X = df.drop(columns=[target_col])
X = pd.get_dummies(X, drop_first=True)
y = df[target_col]

# 4. Split the clean data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Define classification models
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "XGBoost": XGBClassifier(eval_metric='logloss')
}

# 6. Run training loops
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print(f"{name} Test Accuracy: {acc * 100:.2f}%")
```

- **Serialize the Best Model:** Save the trained model and scaler using joblib for loading at Flask application startup.

```
n.py > train_model
import pandas as pd
import joblib
import os
from sklearn.ensemble import RandomForestClassifier

def train_model():
    # 1. Load and clean
    df = pd.read_csv('data/creditcard_data.csv')
    df.columns = df.columns.str.strip()
    if 'Target' in df.columns: df.rename(columns={'Target': 'Approved'}, inplace=True)
    if 'ID' in df.columns: df.drop(columns=['ID'], inplace=True)
    df = df.dropna()

    # 2. Balance classes (Fixes the "Always Rejected" problem)
    df_app = df[df['Approved'] == 1]
    df_rej = df[df['Approved'] == 0].sample(n=len(df_app), random_state=42)
    df = pd.concat([df_app, df_rej])
```

Milestone 4: Web Application Development (app.py)

This milestone focuses on creating , refining the web application for the Credit Card Approval Prediction system using Flask. The trained machine learning model is integrated with the web interface, allowing users to enter applicant details and receive instant approval predictions. The application is designed to provide a simple, reliable, and user-friendly experience. Roots are established for the home page and prediction endpoint.

Activity 4.1: Define the Core Routes in app.py

- Establish routes for the application's two primary endpoints, each serving a distinct purpose:
- / — Home page route that renders the main index.html interface

```python
@app.route('/')
def home():
    return render_template('index.html')
```

- **/predict** — POST endpoint that accepts applicant JSON data, runs inference, and returns the approval result

```python
@app.route('/predict', methods=['POST'])
def predict():
    try:
        data = request.get_json()

        # --- NEW: HARD BUSINESS RULES ---
        income = float(data.get('income', 0))
        years = float(data.get('years_employed', 0))

        if income < 10000:
            return jsonify({'approved': False, 'risk_level': 'High Risk (Invalid Income)'})
        if years < 3:
            return jsonify({'approved': False, 'risk_level': 'High Risk (Invalid Employment)'})
        # --------------------------------

        input_dict = {
            'gender': int(data.get('gender', 0)),
            'age': float(data.get('age', 30)),
            'income': income,
            'years_employed': years,
            'income_type': data.get('income_type'),
            'education': data.get('education'),
            'own_car': int(data.get('own_car', 0)),
            'own_property': int(data.get('own_property', 0))
        }
```

Set Up Route Handlers and Model Loading:

- Load the serialized model and scaler at Flask startup using joblib.load() to avoid reloading on every request.
- Apply input validation: return a 400 error if required fields are missing or contain invalid values.
- Route AJAX requests from the frontend to the /predict endpoint, which processes data and returns a structured JSON response.

```
        # Check what the model actually sees
        print("DEBUG: Final DataFrame for Prediction:")
        print(final_df)
        print("DEBUG: Probability before threshold:", model.predict_proba(final_df)[0][1])
        # Predict
        prob = model.predict_proba(final_df)[0][1]

        is_approved = prob >= 0.20
        risk = "Low Risk" if prob >= 0.15 else ("Moderate Risk" if prob >= 0.12 else "High Risk")

        return jsonify({'approved': bool(is_approved), 'risk_level': risk})

    except Exception as e:
        print(f"DEBUG ERROR: {e}")
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

- Home route: / → renders index.html
- /predict (POST) → accepts JSON, returns { success, result } or { error }

Milestone 5: Frontend Development

Milestone 5 focuses on creating a responsive and interactive user interface for the Credit Card Approval Prediction system. The frontend is designed using HTML, CSS, and JavaScript to provide a simple, user-friendly experience. It allows users to enter applicant details, submit the form, and view the prediction result clearly.

Activity 5.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

. Develop the main index.html page containing the project title, navigation header, applicant details form, and prediction result section.

. Organize the webpage using semantic HTML elements for better readability and maintainability.

. Include suitable icons and modern fonts to improve the visual appearance of the application.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) implementing a clean, professional design consistent with the application's purpose.
- Use flexbox and grid layouts to organise the input form fields and prediction result in a structured, readable layout.
- Add media queries to ensure the layout adapts correctly across desktop, tablet, and mobile screen sizes.

Create Separate UI Components for Each Output Type:

- Prediction Result Card: A prominent card displaying Approved or Rejected with appropriate colour coding (green for approved, red for rejected) and a confidence indicator.
- EDA Visualisation Panel: Embedded plot images (income distribution, occupation chart, correlation heatmap) for data transparency and insight communication.
- Input Form Fields: Clearly labelled form fields for each applicant feature, with appropriate input types and placeholder text.

Activity 5.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Attach a submit event listener to the prediction form that prevents default form submission and sends the applicant data as a JSON POST request to /predict via the Fetch API.
- During inference, disable the submit button and display an animated CSS loader in place of the button text.

Render Results Dynamically:

- Implement the renderResult(data) function in main.js, which dynamically creates and injects DOM elements for the prediction result into its container.

- Apply colour-coded styling to the result card based on the prediction value: green (#27ae60) for Approved, red (#e74c3c) for Rejected.
- After rendering, automatically smooth-scroll the viewport to the results section using scrollIntoView().

```
document.getElementById('predictionForm').addEventListener('submit', async function(e) {
    e.preventDefault();
    console.log("Analyze button clicked!"); // Check console for this

    const resultBox = document.getElementById('result');
    const formData = new FormData(this);
    const formObject = Object.fromEntries(formData.entries());

    try {
        const response = await fetch('/predict', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(formObject)
        });

        const result = await response.json();
        console.log("Server response:", result);

        resultBox.classList.remove('hidden');

        if (result.approved) {
            resultBox.innerHTML = `🎉 Application Approved!<br>Status: <b>${result.risk_level}</b>`;
            resultBox.className = 'result-box success-box';
        } else {
            resultBox.innerHTML = `❌ Application Rejected.<br>Status: <b>${result.risk_level}</b>`;
            resultBox.className = 'result-box danger-box';
        }
    } catch (error) {
        console.error('Fetch error:', error);
    }
});
```

Restore UI State After Prediction:

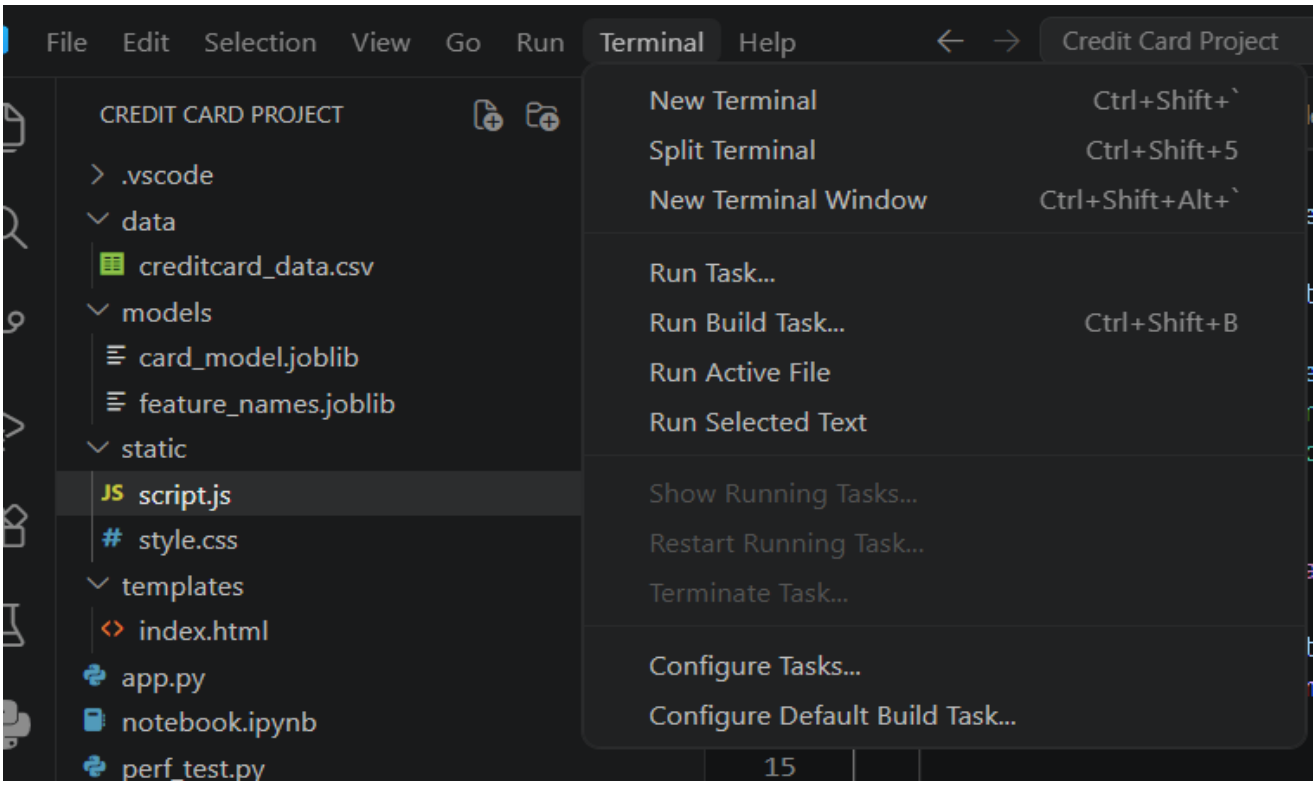
- In the finally block of the async handler, re-enable the submit button and restore the original button text and icon regardless of success or failure.

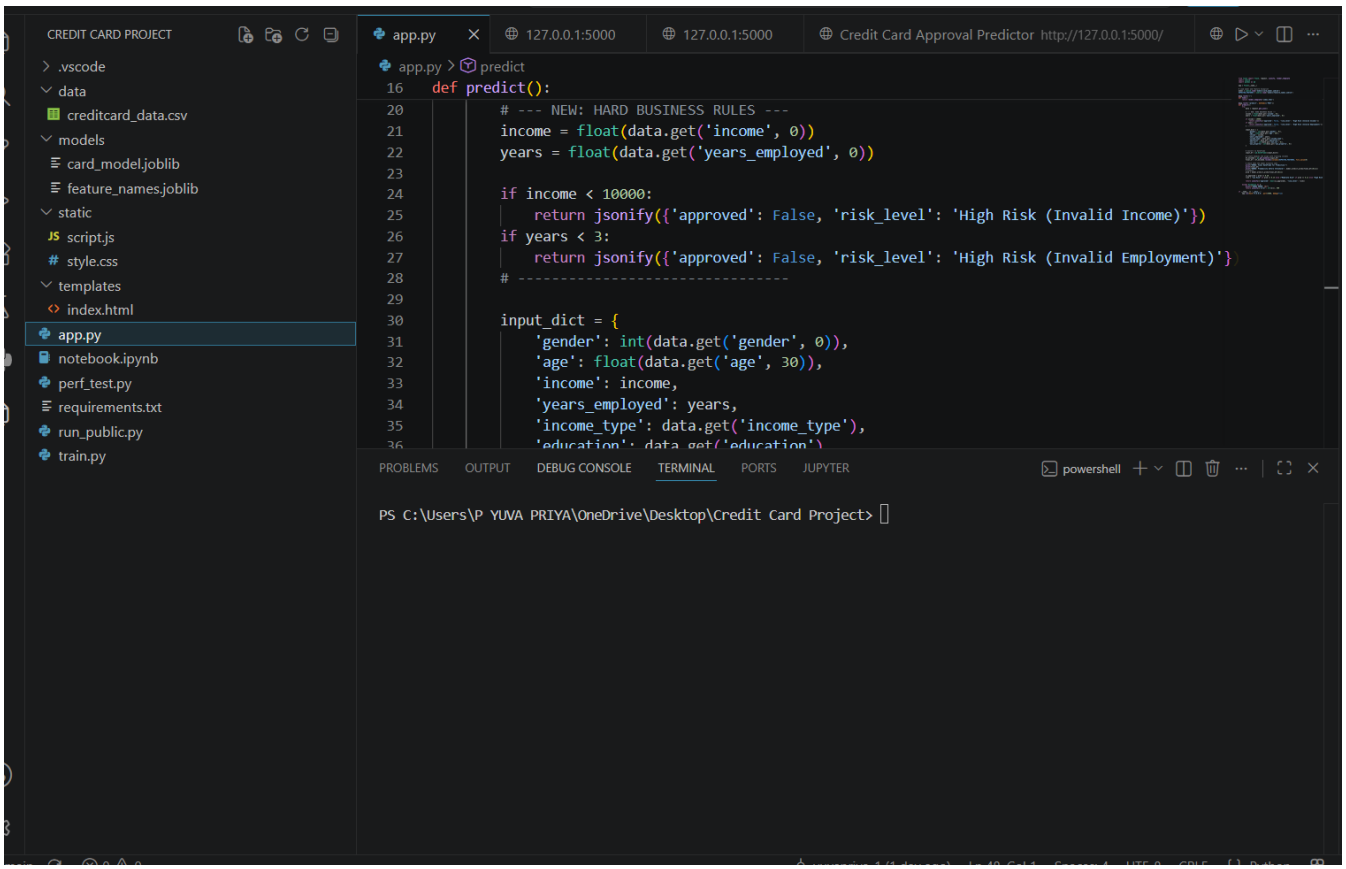
Milestone 6: Deployment

In Milestone 6, the Credit Card Approval Prediction application is deployed by first verifying its functionality in a local environment and then exposing it to the internet using Ngrok. This milestone involves configuring the server environment, installing required dependencies, launching the application, generating a secure public URL through Ngrok, and validating that the application is accessible for demonstration and testing.

Set Up a Virtual Environment:

- Open Terminal and after that select New Terminal.


File Edit Selection View Go Run Terminal Help
Credit Card Project
CREDIT CARD PROJECT
.vscode
data
creditcard_data.csv
models
card_model.joblib
feature_names.joblib
static
script.js
style.css
templates
index.html
app.py
notebook.ipynb
perf_test.py
New Terminal Ctrl+Shift+`
Split Terminal Ctrl+Shift+5
New Terminal Window Ctrl+Shift+Alt+`
Run Task...
Run Build Task... Ctrl+Shift+B
Run Active File
Run Selected Text
Show Running Tasks...
Restart Running Task...
Terminate Task...
Configure Tasks...
Configure Default Build Task...
15


CREDIT CARD PROJECT
app.py
127.0.0.1:5000
Credit Card Approval Predictor http://127.0.0.1:5000/
app.py > predict
def predict():
# --- NEW: HARD BUSINESS RULES ---
income = float(data.get('income', 0))
years = float(data.get('years_employed', 0))
if income < 10000:
return jsonify({'approved': False, 'risk_level': 'High Risk (Invalid Income)'})
if years < 3:
return jsonify({'approved': False, 'risk_level': 'High Risk (Invalid Employment)'}
# ---
input_dict = {
'gender': int(data.get('gender', 0)),
'age': float(data.get('age', 30)),
'income': income,
'years_employed': years,
'income_type': data.get('income_type'),
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER
powershell
PS C:\Users\P YUVA PRIYA\OneDrive\Desktop\Credit Card Project>

- Create and activate a virtual environment, then install all required dependencies from requirements.txt:

```
PS C:\Users\P YUVA PRIYA\OneDrive\Desktop\Credit Card
Project> >python -m venv myenv
PS C:\Users\P YUVA PRIYA\OneDrive\Desktop\Credit Card
Project> >"d:\credit card approval
prediction\venv\Scripts\activate"
PS C:\Users\P YUVA PRIYA\OneDrive\Desktop\Credit Card
Project> >pip install -r requirements.txt
```

Configure Environment Variables:

- Ensure all required environment variables are set. If using external APIs or database connections, create a .env file in the project root and load it using python-dotenv:

```
SECRET_KEY=
3FqpGZZotVveD02pIu5F6uXJe3r_2JbGRmaJE6zMQF1GpK3cr
```

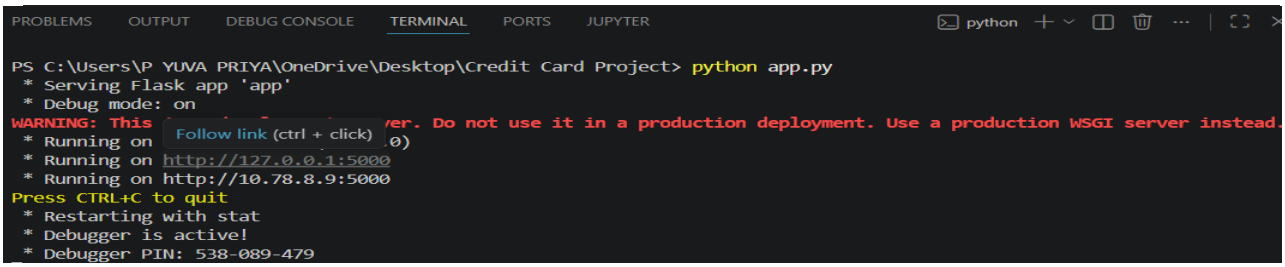
Activity 6.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
PS C:\Users\P YUVA PRIYA\OneDrive\Desktop\Credit Card
Project> >python app.py
```

- Once running, open a browser and navigate to http://127.0.0.1:5000 to access the application



- Interact with the prediction form — test with various applicant profiles across different income levels, occupations, and credit histories to verify the model inference, input validation, and result rendering are all working correctly.

Activity 6.3: Public Deployment via Ngrok

Ngrok creates a secure tunnel from a public URL to your local Flask server, making the application accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
PS C:\Users\P YUVA PRIYA\OneDrive\Desktop\Credit Card
Project> >pip install pyngrok
```

- Download and install the Ngrok client from https://ngrok.com/download.
- Click on the Microsoft Store Installer for the most secure experience.

Configure Your Ngrok Authtoken:

- Sign up at ngrok.com and copy your authtoken from the Ngrok dashboard.
- The app.py script sets the authtoken automatically using:

```
ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")like(1234567890
bcegghfgyuvghijrhijr)
```

- Replace the TOKEN value in app.py with your own Ngrok authtoken before running.

Run the Public Deployment Script:

- Running app.py directly, will start both the Ngrok tunnel and also the Flask server:

```
PS C:\Users\P YUVA PRIYA\OneDrive\Desktop\Credit Card
Project> python app.py
```

- The script opens an HTTP tunnel on port 5000 and prints the live public URL in the terminal:

```
   ========================================================= YOUR PUBLIC WEBSITE URL IS
LIVE! URL: https://captive-actress-exclude.ngrok-free.dev
=========================================================
```

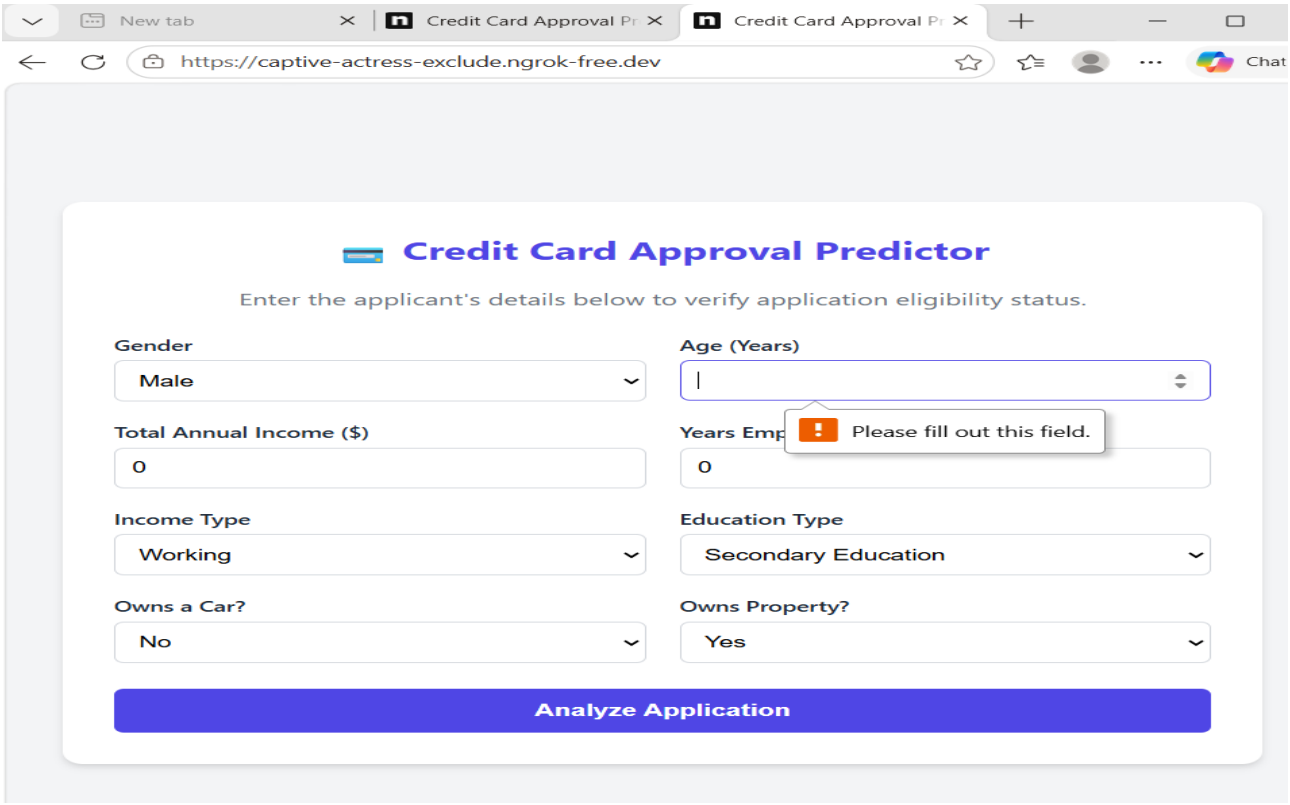
- Share this URL with anyone — they can access the Credit Card Approval Prediction application directly from their browser without any installation.

Important Notes:

- debug=True is required in app.py to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart app.py (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.
- Ngrok free tier sessions expire after a few hours of inactivity. Restart app.py to get a new URL when this happens.

Exploring the Website's Web Pages:

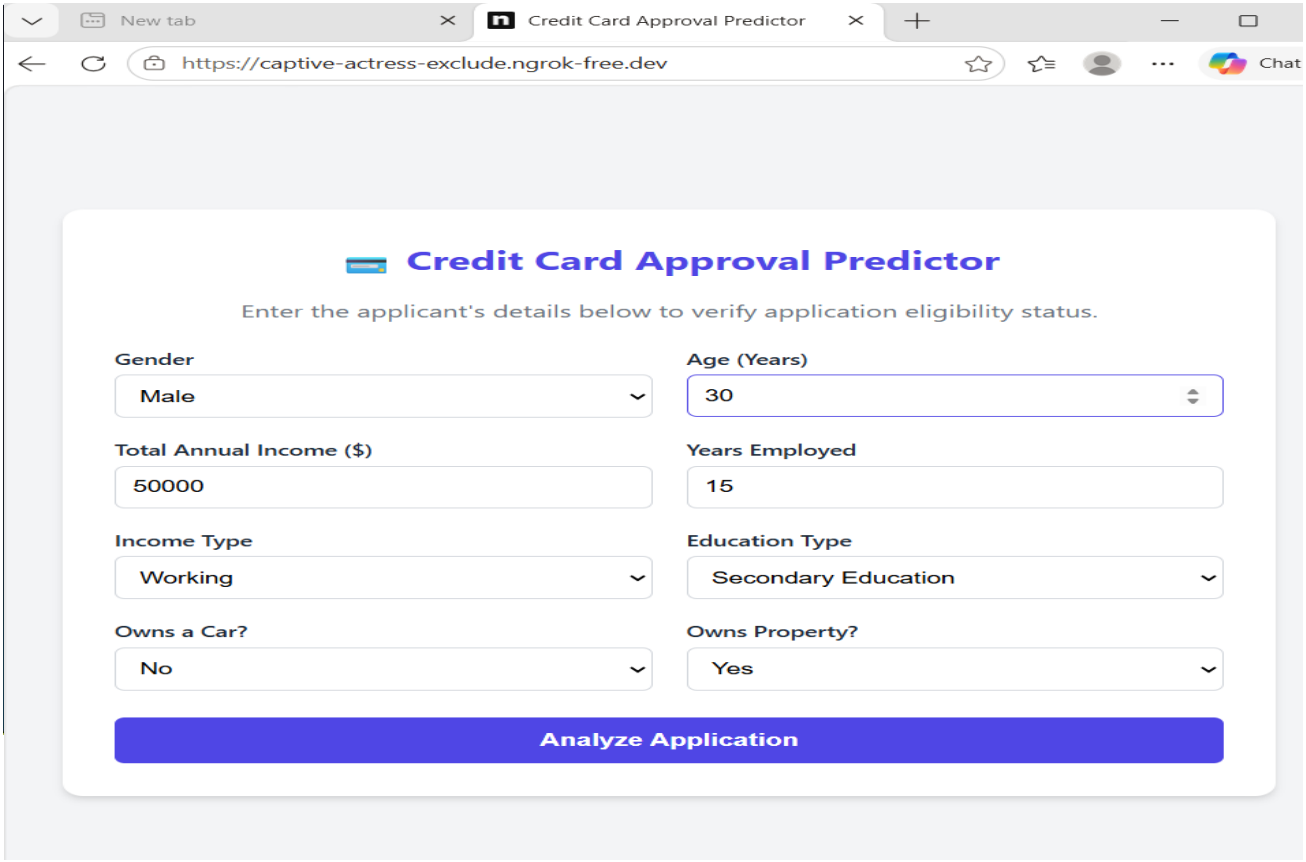
Home / Prediction Page:



Description: The Credit Card Approval Predictor is a web-based machine learning application that evaluates an applicant's eligibility for a credit card based on the information provided. Users enter details such as gender, age, annual income, years of employment, income type, education level, car ownership, and property ownership.After clicking the "Analyze Application" button, the system processes the input using a trained prediction model and displays whether the credit card application is likely to be approved or rejected. The application features a simple, responsive, and user-friendly interface, enabling quick and accurate credit approval predictions while demonstrating the practical use of machine learning in financial decision-making.

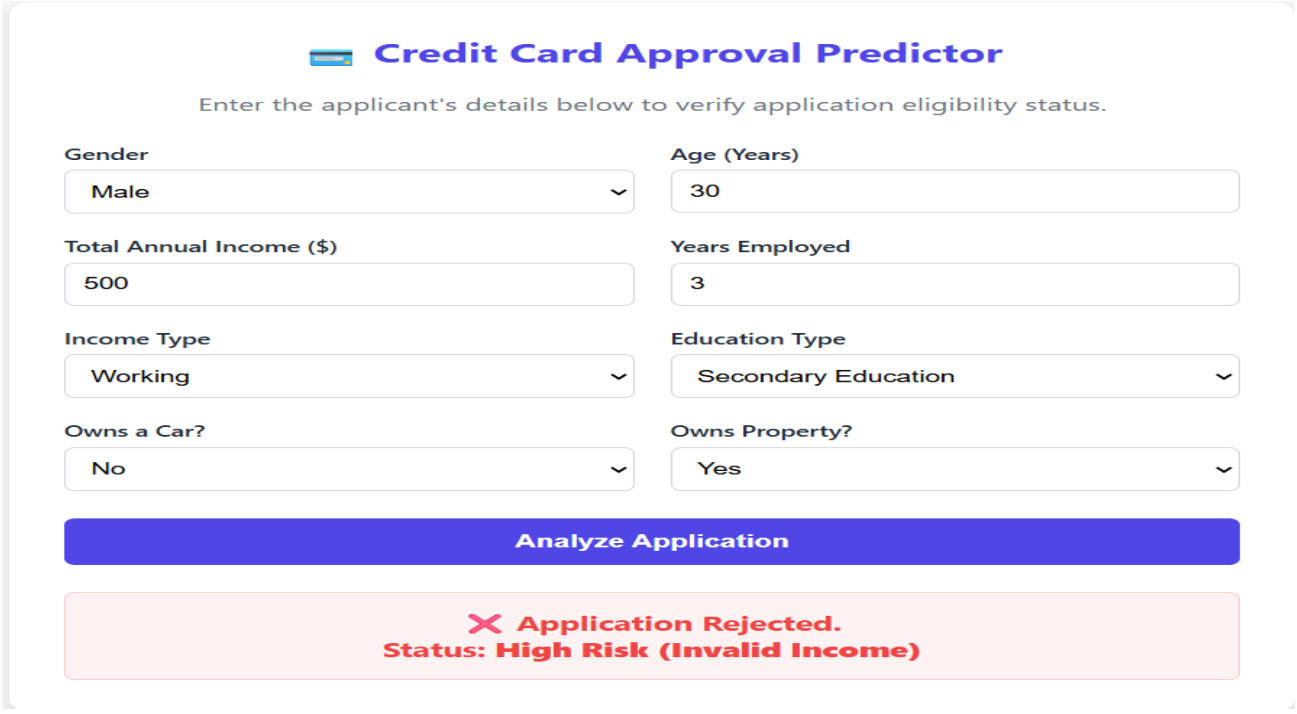
Input Section:

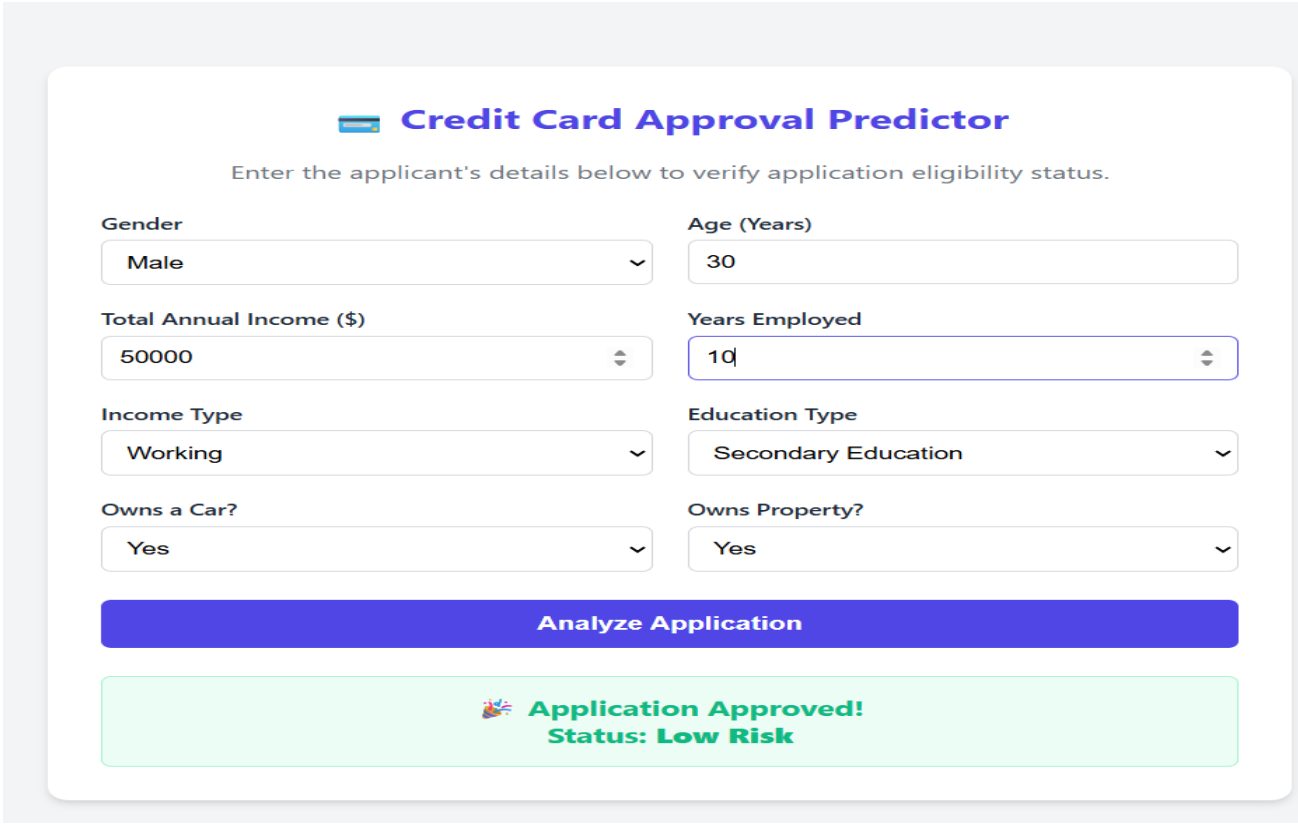
Description: The user enters the applicant's details, including gender, age, annual income, years of employment, income type, education level, car ownership, and property ownership. These inputs are used by the machine learning model to analyze the applicant's financial and personal profile. After clicking the "Analyze Application" button, the system processes the entered information and predicts whether the credit card application is likely to be approved or rejected.



Results Section:

Description: After the user submits the applicant's details, the machine learning model analyzes the provided information and generates the prediction result. The result section displays whether the credit card application is approved or rejected along with the applicant's risk status (such as Low Risk or High Risk). This provides a clear and immediate assessment of the applicant's eligibility based on the entered data..





Project Overview/Summary:

The project includes:

- Developed a machine learning-based web application for credit card approval prediction.
- Predicts whether a credit card application is approved or rejected based on applicant details.
- Uses applicant information such as age, annual income, employment, education, income type, and asset ownership.
- Processes input data using a trained machine learning model to generate predictions.
- Displays the approval status along with the risk level (e.g., Low Risk or High Risk).
- Provides a simple, responsive, and user-friendly interface for easy interaction.
- Helps automate the credit evaluation process, making decisions faster and more accurate.
- Demonstrates the practical application of machine learning in the banking and financial sector.

Objectives:

1.To develop a machine learning-based system for credit card approval prediction.

2.To automate the credit card eligibility assessment process.

3.To analyze applicant details and predict approval or rejection accurately.

4.To reduce manual effort and improve decision-making efficiency.

5.To provide instant prediction results along with the applicant's risk status.

6.To build a simple, responsive, and user-friendly web application.

7.To demonstrate the practical application of machine learning in the banking and financial sector.

8.To improve the speed, consistency, and reliability of credit approval decisions.

Tech Stack/Technologies:

Programming Language

- Python

Data Analysis

- Pandas
- NumPy

Visualization

- Matplotlib
- Seaborn

Machine Learning

- Scikit-Learn

Web Development

- HTML
- CSS
- JavaScript
- Flask

*Deployment

- Render
- Ngrok

*Dataset

- Kaggle (Credit Card Approval Dataset)

*Version Control

- Git
- GitHub

Conclusion:

The Credit Card Approval Predictor is a machine learning-based web application designed to simplify and automate the credit card approval process. By analyzing important applicant details such as age, annual income, employment experience, education level, income type, and asset ownership,

the system predicts whether a credit card application is likely to be approved or rejected. It also provides a risk assessment, helping users understand the decision more effectively.

The project demonstrates the practical application of machine learning in the banking and financial sector by reducing manual effort, improving prediction accuracy, and ensuring faster decision-making. The integration of a user-friendly web interface with a trained machine learning model makes the application easy to use and accessible across different devices.

Looking ahead,this framework can be scaled by incorporating more complex datasets and time financial tracking, allowing for more granular risk profiling. By leveraging advanced predictive analytics, such a system has the potential to foster greater financial inclusivity, offering fair and transparent credit opportunities to a broader demographic of applicants.

Overall, the Credit Card Approval Predictor enhances the efficiency, consistency, and reliability of the credit evaluation process. It serves as a valuable example of how data - driven technologies can support financial institutions in making accurate and timely credit approval decisions while improving customer experience.